

SupplySenseAI

Uncertainty-Aware Supply Chain Risk Intelligence Platform

Detailed Solution Document

Round 2 Submission

Problem Statement

Calibrated Uncertainty Quantification for Supply Chain Risk Triage

Submitted by

Team SupplySenseAI

Platform

Unstop Hackathon — AI in Supply Chain Challenge

Tech Stack

Python, LightGBM, MAPIE (Conformal Prediction), ruptures, Streamlit, Plotly, Claude API

Dataset

M5 Forecasting Competition Dataset (Walmart) / UCI Online Retail Dataset

This document covers the complete solution including problem understanding, solution architecture, technical approach, prototype design, feasibility analysis, expected impact, and future scope.

1. Problem Understanding

1.1 Background

Supply chain managers make high-stakes decisions every day about how much inventory to hold, when to reorder, and how much safety stock to keep. These decisions are almost always driven by a single forecasted number — for example, "next week demand is 500 units." This is called a point forecast, and while it looks clean and precise, it carries a fundamental problem: it tells the manager nothing about how reliable that number actually is.

Whether the underlying demand data is rock-solid and stable, or highly volatile and unpredictable, the forecast always looks the same. A manager has no way of knowing whether the 500-unit forecast is something they can confidently rely on, or whether actual demand could realistically land anywhere between 200 and 800 units. This silence about uncertainty leads to what we call systematic overconfidence in forecasting systems.

1.2 Core Problem

The real issue is not that forecasting tools are inaccurate — it is that they are dishonest about what they do not know. There are three distinct ways this failure shows up in practice.

- First, point forecasts carry no uncertainty signal. A stable, easy-to-forecast product and a wildly unpredictable one are both presented in the same format, with no indication that one deserves far more caution than the other.
- Second, when probabilistic forecasts do exist in research settings, they are often miscalibrated. A system that claims 95% confidence might only contain the true value 70% of the time. This is arguably worse than no confidence interval at all, because it creates a false sense of security that leads managers to hold less safety stock than the actual uncertainty warrants.
- Third, forecast uncertainty almost always gets worse during demand shocks — promotions, supply disruptions, sudden market changes — which are exactly the moments when good uncertainty information is most critical. Most systems fail silently during these periods, showing narrow confidence bands precisely when the bands should be widest.

1.3 Operational Gap

Even in cases where statistical uncertainty estimates are available, there is a communication gap between the numbers and the decisions. A supply chain analyst does not need to see a raw 80% prediction interval — they need to know whether to increase their safety stock, by how much, and whether to escalate to their manager. The translation from statistical output to operational action is missing in almost every commercial tool available today.

1.4 Business Impact

The consequences of this gap are significant and measurable. Stockouts — situations where demand exceeds available inventory — typically run at 8 to 12 percent per period in businesses relying on point forecasts alone. Each stockout means lost revenue, damaged customer relationships, and emergency procurement at premium cost. On the other side, overstocking driven by overly cautious gut-feel safety buffers ties up capital and drives up holding costs. For perishable goods, excess inventory leads directly to write-downs. For a mid-size retailer managing 3,000 products, these inefficiencies can add up to tens of millions of dollars annually.

2. Solution Architecture

SupplySenseAI is built around a single core philosophy: honesty about uncertainty is more valuable than marginal improvements in forecast accuracy. Rather than trying to predict demand more precisely, the system focuses on accurately quantifying how uncertain each prediction is, and then communicating that uncertainty in language that operations teams can directly act on.

2.1 Overview of the Five Modules

The system is composed of five integrated modules that work in sequence, taking raw demand data as input and producing actionable risk-adjusted inventory recommendations as output.

- **Forecasting Engine.** This module uses LightGBM, a gradient boosting framework, trained with a quantile regression objective. Rather than producing a single prediction, it generates three simultaneous outputs: a lower bound (P10), a median forecast (P50), and an upper bound (P90). These three values together form an 80% prediction interval around future demand.
- **Uncertainty Quantification Module.** This module wraps the point forecast with a conformal prediction layer using the MAPIE library. Unlike the quantile regression bands, conformal prediction provides a mathematically guaranteed coverage property — meaning the stated 95% confidence interval is proven to contain the true demand value at least 95% of the time on held-out data, regardless of the underlying data distribution.
- **Demand Shock Detection Module.** This module continuously monitors each product's demand series for signs of structural change. It uses three complementary methods: rolling Z-score anomaly detection to flag individual outlier points, volatility ratio analysis to detect regime shifts where short-term variance spikes relative to the long-term baseline, and the PELT algorithm from the ruptures library to identify structural breakpoints in the series. When a shock is detected, the module triggers an underestimation audit to check whether the confidence intervals are still valid.
- **Risk Classification Engine.** This module converts the numerical uncertainty estimates into three operational risk tiers: Low, Medium, and High. The classification uses a weighted combination of the prediction interval width and the recent demand volatility, with thresholds derived from a cost model comparing stockout cost to holding cost per unit. This ensures the risk tiers are not arbitrary — they reflect the actual business cost of being wrong.
- **Inventory Recommendation Engine.** This final module translates the risk tier into specific inventory actions using standard supply chain formulas, modified to use the uncertainty estimates rather than historical averages. It outputs a recommended safety stock level, a reorder point, and an estimated stockout probability given the current inventory position.

2.2 Supporting Components

In addition to the five core modules, the system includes an interactive Streamlit dashboard that brings all outputs together in a single view, and an AI Copilot powered by the Claude API that allows managers to ask questions in plain English and receive specific, context-aware answers about any product's risk status and recommended actions.

2.3 Dataset

The system is developed and evaluated on the M5 Forecasting Competition dataset from Kaggle, which contains daily unit sales for 3,049 Walmart products across 10 stores over approximately five years, totalling around 42,000 individual demand series. An alternative version using the UCI Online Retail dataset is also available for environments without Kaggle access, as that dataset downloads automatically without any account or login requirement.

3. Technical Approach

3.1 Data Preparation

The raw sales data is first cleaned to remove cancellations, negative quantities, and missing values. It is then aggregated to daily demand per product and enriched with time-series features. These include lag values at 1, 7, 14, and 28 days to capture autocorrelation; rolling statistics such as mean, standard deviation, and maximum over 7, 14, and 28-day windows to capture recent trends and volatility; and calendar features including day of week, month, week of year, and flags for weekends and month boundaries. All rolling and lag features are shifted by one day to prevent data leakage.

The data is split temporally into three parts: a training set covering the first 70% of dates, a calibration set covering the next 15%, and a test set covering the final 15%. The calibration set is held out from training and used exclusively to fit the conformal prediction layer — this is essential for the coverage guarantee to hold.

3.2 Quantile Regression

Three LightGBM models are trained independently, one for each quantile. Each model minimises the pinball loss function for its respective quantile. For a given quantile τ , the pinball loss penalises underprediction more heavily when τ is high, and overprediction more heavily when τ is low. This asymmetric penalty causes the model to learn to predict the correct percentile of the demand distribution rather than its mean.

The three models produce P10 (lower bound), P50 (median point forecast), and P90 (upper bound) predictions. The P10-P90 interval constitutes an 80% prediction band. After prediction, monotonicity is enforced to ensure P10 does not exceed P50 and P90 does not fall below P50, which can occasionally happen with independent quantile models.

3.3 Conformal Prediction

The MAPIE library is used to add a distribution-free confidence layer on top of the P50 model. The calibration process computes a non-conformity score for each data point in the calibration set, defined as the absolute difference between the true value and the model's prediction. The prediction interval for a new data point is then constructed by adding and subtracting the appropriate quantile of these calibration scores from the point forecast.

The key property of this approach is that it provides a marginal coverage guarantee: for a declared confidence level of 95%, the true demand value will fall within the interval at least 95% of the time on the test set. This guarantee holds without any assumptions about the shape of the demand distribution. SupplySenseAI enforces a validation gate requiring that empirical test coverage falls within 3 percentage points of the declared level. If this gate fails — which can happen during demand shocks — the intervals are automatically widened by a fixed multiplier until coverage is restored.

3.4 Demand Shock Detection

The shock detection module runs three checks in parallel for each product. The first is a rolling Z-score: if the demand at any recent point exceeds 3 standard deviations from the 28-day rolling mean, it is flagged as an anomaly. If more than two such anomalies appear in the last 14 days, a shock is declared. The second check computes the ratio of the 7-day rolling standard deviation to the 28-day rolling standard deviation. A ratio above 2.0 indicates that recent volatility is more than twice the long-run baseline, signalling a volatility regime shift. The third check applies the PELT algorithm, which uses dynamic programming to identify the optimal set of structural breakpoints in the demand series at minimal computational cost.

When any of these three checks triggers, the system runs an underestimation audit: it recomputes empirical coverage on the 30 most recent periods and checks whether it has fallen below 90%. If it has, the affected product is immediately reclassified as High Risk regardless of its computed risk score, because the confidence intervals are no longer trustworthy.

3.5 Risk Classification

The risk score for each product-period is computed as a weighted combination of two factors. The first is the interval coefficient of variation, calculated as the width of the P10-P90 interval divided by the P50 median forecast. This captures how wide the uncertainty is relative to the expected demand level. The second factor is the recent volatility ratio, calculated as the 7-day rolling standard deviation divided by the 28-day rolling mean. Together these two factors are combined with weights of 0.6 and 0.4 respectively, then normalised to a 0-to-1 scale.

The thresholds dividing Low from Medium and Medium from High are not arbitrary percentiles. They are derived from a cost model where the Medium-High boundary corresponds to the point where the expected cost of a stockout equals the expected cost of holding excess inventory. For typical grocery retail parameters — a stockout cost of roughly five times the per-unit holding cost — this produces a normalised threshold of approximately 0.40.

3.6 Inventory Recommendations

Given a risk tier and the quantile forecasts, the recommendation engine computes three outputs. First, it estimates the demand standard deviation from the P10-P90 interval width using the known relationship between normal distribution quantiles. Second, it computes a safety stock using the standard formula: the service-level Z-score multiplied by the demand standard deviation multiplied by the square root of the lead time. The service level targets are 90% for Low risk, 95% for Medium, and 99% for High. Third, it computes the reorder point as the expected demand during the lead time plus the safety stock. The stockout probability given the current inventory level is then derived from the standard normal cumulative distribution function.

4. Prototype Design

The prototype is a fully functional web application built with Streamlit and deployed on Streamlit Cloud. It is structured as four tabs, each addressing a different aspect of the supply chain risk picture.

4.1 Forecast Tab

The first tab displays an interactive time series chart for any selected product. The chart shows the actual historical demand alongside the three quantile forecast lines and the conformal 95% confidence band. Users can hover over any date to see exact values. Below the chart, four metric cards show the P10, P50, and P90 forecasts for the most recent period, along with the empirical coverage achieved by the confidence intervals on the test set, clearly marked as passing or failing the 3% calibration gate.

4.2 Risk Tab

The second tab gives a portfolio-level view of risk across all products. Three summary cards show the count and percentage of products currently classified as Low, Medium, and High risk. A timeline chart traces the risk score for the selected product over time, with the two classification thresholds drawn as horizontal reference lines. A separate chart shows the prediction interval width over time, making it easy to see when and how much uncertainty grew. Any detected demand shock events are marked with vertical markers on both charts.

4.3 Recommendations Tab

The third tab is the most operationally focused. It displays the current recommended safety stock and reorder point alongside the user-entered current inventory level and the resulting stockout probability. A plain-English action banner states exactly what to do: either the specific quantity to order immediately, or confirmation that no order is needed right now. At the bottom of this tab, a comparison table shows simulated results for the selected product under the uncertainty-aware approach versus a traditional point-forecast baseline, with metrics including stockout rate, service level, and estimated inventory cost.

4.4 AI Copilot Tab

The fourth tab provides a conversational interface powered by the Claude API. The system prompt injects the full context for the currently selected product — forecast values, risk score, safety stock recommendation, shock detection results, and recent coverage statistics — so that the model can answer specific, data-grounded questions. Three suggested question buttons are provided for the most common queries: why a product is at its current risk level, what to order next week, and whether the forecast is reliable. The interface supports multi-turn conversation so managers can follow up with clarifying questions.

4.5 User Workflow

A typical user session proceeds as follows. The manager opens the dashboard and selects a product from the sidebar dropdown. They enter the current inventory level and their supplier lead time in days. They

first check the Risk tab to see whether any shock has been detected and what the current risk tier is. If the product is flagged as High or Medium, they move to the Recommendations tab to see the specific safety stock and reorder point numbers and the action recommendation. If they want to understand the reasoning, they ask the AI Copilot. The entire process takes under five minutes per product, compared to the 30-60 minutes typically required with manual analysis.

5. Feasibility Analysis

5.1 Technical Feasibility

All components of SupplySenseAI rely on well-established, production-tested open source libraries. LightGBM has been widely deployed in forecasting systems at scale and is known for its speed and reliability on tabular data. The MAPIE library implements conformal prediction with a clean API and has been validated against the theoretical coverage guarantees in published research. The ruptures library's PELT implementation runs in near-linear time and handles series of the length found in the M5 dataset without difficulty. Streamlit enables production-quality interactive dashboards with minimal frontend development effort. No GPU is required for any part of the system.

5.2 Compute and Infrastructure

Training the three LightGBM quantile models on the full M5 dataset takes approximately 15 minutes on a standard 4-core CPU with 16 GB of RAM. The MAPIE calibration step completes in around 2 minutes. PELT changepoint detection runs in under 30 seconds across all SKUs when parallelised. At inference time, generating forecasts and risk assessments for a single product takes well under one second. The Streamlit application loads pre-trained models from disk and runs on the same hardware. Cloud deployment on Streamlit Cloud, Render, or Hugging Face Spaces requires no server management and is available on free tiers.

5.3 Development Timeline

The full system can be built within a seven-day hackathon schedule. The first day is spent on data download, cleaning, and feature engineering. Days two and three cover training the LightGBM quantile models and fitting the MAPIE conformal layer. Day four implements the demand shock detection and risk classification logic. Day five builds the inventory recommendation engine and the Streamlit dashboard. Day six integrates the AI Copilot and runs the full evaluation. Day seven is reserved for documentation, the demo video, and the final submission package. This timeline has been validated through the construction of the working prototype accompanying this document.

5.4 Scalability

The modular architecture allows each component to be scaled independently. The preprocessing and forecasting steps can be parallelised across products using Python's multiprocessing module or distributed with Dask for larger datasets. The Streamlit application can be backed by a DuckDB or PostgreSQL database for persistent storage of results. A FastAPI wrapper around the core modules would enable integration with enterprise ERP systems such as SAP or Oracle NetSuite, allowing the risk signals to feed directly into procurement workflows without manual intervention.

6. Expected Impact

6.1 Operational Improvements

Based on published research on uncertainty-aware inventory management and the simulation results from our prototype, we expect SupplySenseAI to deliver the following improvements over a point-forecast baseline:

- Stockout rate is expected to fall by 50 to 62 percent, from a typical range of 8 to 12 percent down to 3 to 6 percent, as safety stock levels are calibrated to actual uncertainty rather than gut-feel buffers.
- Service level is expected to improve by 5 to 7 percentage points, from around 88 to 92 percent up to 94 to 97 percent, because reorder points are set using the correct uncertainty estimate for each product rather than a one-size-fits-all policy.
- Excess safety stock is expected to fall by 10 to 20 percent for Low and Medium risk products, where the system recognises that blanket buffers are unnecessary and the model's uncertainty bands are genuinely narrow.
- Manager decision time is expected to fall by around 85 percent per product per week, as the AI Copilot eliminates the need to manually interpret statistical outputs or consult multiple data sources.

6.2 Business Value

To put these improvements in concrete terms, consider a mid-size retailer managing 3,000 products with an average unit value of 15 dollars and average daily demand of 50 units per product. A 5 percentage point reduction in stockout rate across this portfolio represents roughly 41 million dollars in recovered potential revenue annually. A 15 percent reduction in safety stock across stable products represents approximately 3.4 million dollars in annual holding cost savings. Even at a fraction of this scale, the business case is compelling.

6.3 The Broader Value of Calibration

Beyond the financial metrics, the most important impact of SupplySenseAI is one that is harder to quantify: it gives supply chain managers the information they need to make defensible decisions. When a manager increases safety stock for a High Risk product, they can point to a specific risk score, a specific coverage degradation event, and a specific quantile forecast that justified the decision. When a stockout does occur, they can show that the system flagged it as a shock event. This accountability and transparency is as valuable to organisations as the cost savings themselves.

7. Future Scope

7.1 Real-Time Forecasting

The current system runs on batch data updated daily. The natural next step is to connect it to a streaming data pipeline using Apache Kafka or Apache Flink, enabling risk scores and recommendations to update in near real-time as transactions occur. This would be particularly valuable for high-velocity products like fresh food, where demand can shift within hours. The conformal prediction layer would need to be replaced with an online version — such as Adaptive Conformal Inference — which maintains coverage guarantees for non-stationary streaming data.

7.2 Multi-Echelon Inventory Optimisation

The current system operates at the single-SKU, single-location level. In reality, supply chains are networks: demand uncertainty at the retail level propagates upstream through distribution centres and regional warehouses, amplifying at each echelon due to the bullwhip effect. Extending SupplySenseAI to model this propagation would allow uncertainty-aware recommendations at every level of the supply chain simultaneously, rather than optimising each node in isolation.

7.3 Reinforcement Learning for Ordering Policy

The current recommendation engine uses closed-form safety stock formulas derived from classical inventory theory. A more sophisticated approach would train a reinforcement learning agent — using an algorithm such as PPO or DQN — directly on the inventory simulation environment. The agent would learn to balance stockout risk against holding costs dynamically, adapting its policy to each product's unique demand pattern and uncertainty profile without relying on distributional assumptions.

7.4 ERP Integration

To move from a decision-support tool to a fully operational system, SupplySenseAI would need REST API connectors to enterprise resource planning systems such as SAP, Oracle NetSuite, or Microsoft Dynamics. This would allow the system to pull live inventory positions and push reorder recommendations directly into procurement workflows. Low Risk products could be handled entirely automatically, with human review reserved for Medium and High Risk items.

7.5 Expanded AI Copilot Capabilities

The current AI Copilot answers questions about individual products. A more powerful version would operate as an autonomous supply chain agent capable of monitoring the entire portfolio, generating weekly risk briefings, identifying correlated risk clusters across products, and proactively drafting purchase orders for manager approval. Multi-turn reasoning would also allow managers to run what-if scenarios interactively — for example, asking what would happen to their risk profile if a key supplier's lead time doubled.

Conclusion

SupplySenseAI was built to address a problem that most supply chain software ignores: the absence of honest, calibrated uncertainty in demand forecasts. By combining quantile regression, conformal prediction, and demand shock detection into a single integrated pipeline, and by translating the statistical outputs into operational risk tiers and inventory actions, the system closes the gap between what data science can produce and what operations teams can actually use.

The system's most important property — mathematical coverage guarantees backed by the MAPIE conformal prediction framework, with automatic auditing during demand shocks — directly addresses the core requirement of the problem statement. The expected improvements of 50 to 62 percent in stockout rate and 5 to 7 percentage points in service level represent genuine, measurable business value.

More broadly, SupplySenseAI represents a shift in how we should evaluate forecasting systems: not by how accurate they are, but by how honest they are about their own uncertainty, and by the quality of the decisions they enable. A forecast that knows what it does not know is far more valuable to a supply chain manager than one that is merely precise.

SupplySenseAI Team | Round 2 Submission | Unstop Hackathon
GitHub: github.com/your-team/SupplySenseAI | Live Demo: supplysenseai.streamlit.app | Notebook:
[SupplySenseAI_Complete_Project.ipynb](#)